— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 5

7 Concurrent and Distributed Systems (djg11)

syllabus keywords

(*a*) What are the distinguishing differences between a process and a thread? How are they best mapped to processors? [4 marks]

Answer: A process is a virtual address space secured against other processes by the O/S. Threads within a process typically use the process space with one stack per thread but with shared static program and data and shared heap. It is best if the O/S understands threads and can map them to separate cores to exploit parallel processing, but processor affinity, either managed by the O/S or explicitly by the application will be best for caching. *Four distinct points needed for four marks.*

monitor, condition variable

(*b*) (*i*) What are the main features of a *monitor* in concurrency theory? [2 marks]

Answer: A simple monitor is a collection of methods that typically access some common shared data. Entry to the monitor is guarded by a mutex such that only one thread is inside the monitor at once: the remainder are queued on that mutex.

(*ii*) How can a thread suspend itself inside a monitor? Use a state diagram or flow chart to explain your answer. [5 marks]

Answer: A thread inside a monitor can suspend itself on a condition variable. Only one is needed but having multiple condition variables reduces unnecessary wake-ups and simplifies dispatch code.

The requested diagram must show the external thread arriving at a first state where it spins trying to acquire the monitor lock. Once inside we have a succession of states where some exit the monitor and release the monitor lock as they leave and at least one arc waits on the condition variable. Other paths may signal the condition variable but full marks if this is not shown.

A feature that must be present in the diagram is a transition into and then out of a pause state for the CV where the input arc is somehow labelled as `wait` and the output is guarded with a response to `signal`, `signal_all` or `broadcast` etc. by another thread. These two arcs must be further annotated to show the simultaneous release and re-acquire of the monitor lock as they are respectively taken. A full solution will check that the condition does indeed hold and branch back to the wait state if it does not (the common `while` construct around a CV wait).

priority inversion

(*c*) Define *priority inversion* and give an example of a situation where it may occur. [3 marks]

Answer: Priority inversion occurs when a lower-priority process runs and stops higher-priority work that is runnable from being run. A first principle is that low-priority work must not hold locks for long periods, otherwise priority will not be effective. But where a mid-priority task preempts the low-priority task while it holds the lock the principle is violated and the composition does block the high-priority work.

(*d*) Several threads split over several processes all write lines of text to one output device. Characters from different lines must not be interleaved. The processes

have different priorities. Outline the major software components (such as processes, threads, buffers, locks and system calls) involved in this task and how they interact. Identify the points where threads will block and examine whether priority inversion is likely to happen. [6 marks]

Answer: The threads will each call a `print` routine that acquires a lock on a per-process output buffer before appending the line to the buffer. Assume all threads within a process have equal priority, so no localised inversion is possible.

When a per-thread buffer has a suitable amount of content (eg. several lines or at least one line and 250 ms have passed or there is insufficient buffer space for a newly-arrived, pre-thread write), it will invoke a write system call. The system call always transfers an integral number of lines.

The output device (or output file) will have its own write buffer and the system call write data will be copied in if space exists. This copy needs to be atomic w.r.t. other such writes (trivial in a single-threaded kernel otherwise locks are needed).

Processes and threads will be yielding at any point where there is insufficient space for them to complete their write. We assume the ouput device buffer is drained asynchronously by interrupt. *3 marks for software design (this is not an O/S course so system call aspect may be neglected).*

If a low-priority task makes a lot of writes so that the outut device throughput is saturated, the progress of the higher-priority tasks will tend to be blocked unless we implement mechanisms for priority-based load balancing on the output device. This is not textbook priority inversion, it is just poor system dimensioning.

Under normal circumstances, all of the write data copy operations will be very quick or else the thread/process will yield *without holding any lock*. Therefore we do not have the typical prerequisites for priority inversion and I cannot see it happening. *3 marks for determination.*
